

Smart City Traffic Intelligence: From Data Analytics to AI-Powered Mobility

Final Capstone Report

Dataset: Metro Interstate Traffic Volume — ~48,000 hourly records, westbound I-94, Minneapolis-St Paul (2012-2018), with weather and US federal holiday data.

1. Executive Summary

Across all three parts of this capstone — SQL/statistical analysis, a reproducible Python pipeline, and machine learning/AI — one finding recurs consistently, confirmed independently by at least five different analytical methods: **this corridor's traffic is overwhelmingly driven by a stable, repeating time-of-day and day-of-week pattern, not by weather.** A regression model using only cyclical hour/day features (no weather, no recent-traffic memory) explains 94% of the variance in traffic volume; SHAP analysis of a comparable model confirms recent traffic and time-of-day dominate feature importance; and the probability/correlation analysis in Part 1 independently found the same weak weather effect from first principles. This has direct operational implications: **traffic management interventions should be time-based, not weather-triggered.**

2. Methodology by Part

Part 1 — Data Analytics

Raw data was loaded into SQLite and analysed with SQL (yearly trends, holiday temperature comparisons), descriptive statistics and correlation (temperature vs. traffic volume), and probability/conditional probability (congestion vs. weather independence, odds ratios). A Power BI dashboard adds interactive visual exploration on top of the same cleaned data. See `part1_data_analytics/insights_report.md` for full findings.

Part 2 — Python Pipeline

A reproducible pipeline (`pipeline.py`) loads, validates the schema of, and cleans the raw CSV — standardising inconsistent weather-description casing, parsing/validating timestamps, removing 17 exact duplicates and collapsing 7,612 duplicate-timestamp rows, and imputing two classes of physically impossible sensor readings (OK temperature, >9,000mm hourly rainfall) using month-specific medians. `feature_engineering.py` then builds an ML-ready feature set (cyclical time encodings, weather indicators, scaled numerics, a data-driven congestion category), `visualizations.py` produces 4 Matplotlib charts, and `cli_app/app.py` exposes the processed data through a 4-command CLI. Every step is logged via Python's logging module (`never print()` for internal status), with a full sample trail in `part2_python/pipeline.log`. See `part2_python/report.md`.

Part 3 — Machine Learning and AI

Building directly on Part 2's featured dataset, Part 3 developed: two classification algorithms (Logistic Regression, Random Forest) predicting a documented proxy accident-risk label; two regression algorithms (Linear Regression, Gradient Boosting) predicting traffic volume; K-means clustering of traffic conditions and association rule mining of congestion patterns; an LSTM sequence model for traffic volume with SHAP explainability applied to a comparable tree-based model; MLflow experiment tracking across every model trained; a model-driven travel-timing recommendation system; a FastAPI deployment simulation and drift-monitoring/alerting mechanism; and a responsible AI report covering bias, fairness, governance and sustainability. See `part3_machine_learning/README.md` and the individual task write-ups referenced there.

3. Key Findings (synthesised across all three parts)

1. **Time-of-day/day-of-week is the dominant driver of traffic volume**, confirmed by: the SQL/probability analysis (Part 1), the visualisation of weekday-vs-weekend patterns (Part 2), the high R^2 of a time-features-only regression model (Part 3 Task 1), and SHAP feature importance ranking hour/lag features above weather (Part 3 Task 3).
2. **Weather has a real but modest effect**, never the primary signal: temperature-traffic correlation $r \approx 0.14$ (Part 1), weather category accounts for a much smaller swing in average volume than hour-of-day (Part 2), and weather features rank well below time/lag features in every Part 3 model.
3. **Data quality required active handling, not assumption**: uneven yearly hour-coverage (a genuine sensor/collection gap, not a real traffic pattern), duplicate timestamps from overlapping weather observations, inconsistent text casing, and physically impossible sensor readings were all identified and handled with documented, auditable logic rather than silently dropped or ignored.
4. **The proxy accident-risk label is a congestion/weather indicator, not a validated accident predictor** — its very high classification metrics partly reflect its own construction (from weather + congestion) and should not be over-interpreted; this is stated explicitly and repeatedly (Part 3 Task 1, `responsible_ai_report.md`) so it isn't misread as more than it is.
5. **A simpler model can be the responsible choice**: the lag-feature Random Forest slightly out-performed the LSTM while being far cheaper to train — a concrete instance of the sustainability principle that model complexity should track actual accuracy gain, not novelty.

4. Recommendations for the Smart City Mobility Analytics Team

- Prioritise **time-based** interventions (signal timing, transit scheduling, congestion pricing windows) over weather-triggered ones.
- Treat the accident-risk proxy as a **congestion/weather severity indicator** for internal planning use only, until validated against real incident records — do not present it publicly as an accident forecast.
- Adopt the **travel-timing recommendation system** (Part 3, Task 5) as a low-cost, immediately actionable public-facing tool; it is directly derived from the validated regression model's outputs.
- Put the **monitoring/alerting simulation** (Part 3, Task 6) into an actual scheduled job with a named owner before relying on any model operationally — a monitoring script that isn't run isn't monitoring.
- Investigate the **sensor/data-collection gaps** identified in Part 1 directly; they affect the reliability of any year-over-year comparison drawn from this dataset.

5. Repository Guide

See each part's own README for structure and run instructions: `part1_data_analytics/` (`insights_report.md`, `powerbi/traffic_dashboard.pbix`), `part2_python/README.md`, `part3_machine_learning/README.md`.